

TEAR

Product Teardown Platform

Tech Stack . Data Flow . Architecture

DOCUMENT TYPE	Technical Architecture Reference
PLATFORM	Next.js 14 . TypeScript . Anthropic Claude API
PHASE	Phase 1 - Core Pipeline
GENERATED	June 2026

TEAM

Ashutosh	Lead Dev . Backend agents, API routes, Supabase schema, Vercel deploy
Ashish	Architecture & UX . Designs, project context, teardown quality
Nishanth	QA & Content . End-to-end testing, PPTX validation, prompt engineering

01

Project Overview

Tear is an AI-powered product analysis platform. Users enter any software product name, complete a structured Q&A (Tier 1 intake), and receive a citation-backed research document plus a downloadable PowerPoint deck - all generated by a multi-agent pipeline powered by Anthropic's Claude API.

The platform is built on Next.js 14 (App Router) and is deployed on Vercel. It uses Server-Sent Events (SSE) for real-time streaming of agent progress to the client, Supabase (PostgreSQL) for session and research persistence, and Zustand for client-side state management backed by localStorage.

> Phase 1 scope: Landing > Intake Q&A > Live Agent Feed (SSE) > Research Doc > PPTX > Helper Chat. No authentication. Session-based via localStorage UUID.

9

Total Routes

3 Live

Agent Stages

Design system: Warm off-white background (#FDFAF6) . Terracotta red primary (#C2451E) . Near-black text (#1C1412)
. Lora serif headings + DM Sans body

02

Full Tech Stack

FRONTEND FRAMEWORK

- . **Next.js 14** App Router, SSR/SSG, file-based routing, API routes as serverless functions
- . **React 18** UI rendering, hooks, suspense boundaries
- . **TypeScript 5** Full type safety across all layers

STYLING & UI

- . **Tailwind CSS 3** Utility-first styling with custom design tokens (tear-bg, tear-primary, etc.)
- . **Framer Motion 12** Page transitions, component animations, fade-up effects
- . **Lucide React** Icon system
- . **clsx + tailwind-merge** Conditional class utilities

STATE MANAGEMENT

- . **Zustand 5** Global session store persisted to localStorage via persist middleware

AI & AGENTS

- . **Anthropic Claude API** claude-haiku-4-5 for Question/Crawler agents . claude-sonnet-4-6 for Document/Deck agents
- . **@ai-sdk/anthropic 3** Vercel AI SDK wrapper for Anthropic - generateText, streamText, tool use
- . **ai 6 (Vercel AI SDK)** Core streaming primitives, useChat hook
- . **webSearch_20250305** Anthropic's built-in web search tool (no third-party crawler APIs)

DATABASE & BACKEND

- . **Supabase (PostgreSQL)** Session storage, research docs, agent logs - tables: sessions, teardown_research, teardown_qa, teardown_pptx, agent_logs
- . **@supabase/supabase-js 2** With anon client (browser) and admin client (service role key for server routes)

DATA FETCHING

- . **tRPC 11** Type-safe API layer between client and server
- . **@tanstack/react-query 5** Server state management, caching, background refetch

DOCUMENT GENERATION

- . **@react-pdf/renderer 4** PDF generation - ResearchPdfDoc component renders each section as an A4 page
- . **PptxGenJS 4** PowerPoint deck generation from DeckData JSON structure

CHARTING & VISUALISATION

- . **Recharts 3** Research data charts in the deck and research views
- . **Chart.js 4** Canvas-based charts for pipeline stats

INFRASTRUCTURE

- . **Vercel** Deployment, serverless functions, Edge runtime for SSE routes
- . **Node.js 18+** Runtime for API routes

03

User Journey & Page Flow

The application is structured as a linear multi-step flow. Each step is a dedicated Next.js App Router page. Session state (product name, Tier 1 answers, research doc, deck data) is held in Zustand and persisted to localStorage under the key 'tear-session'. A UUID session ID is generated on first load and used as the URL parameter for all downstream pages.

/ . Landing

Hero section with product search input. On submit, productName is written to Zustand store and user is pushed to /intake.

/intake

3-step Tier 1 Q&A:

Step 1 - Dimensions (multi-select: UX, Revenue, Tech, GTM, Community, Financials)

Step 2 - Research goal (Competitive, PM learning, Investor, Curiosity)

Step 3 - Depth (Quick overview, Full teardown, Deep dive)

On complete, tier1Answers saved to store and user pushed to /pipeline/[sessionId].

/pipeline/[sessionId]

Opens SSE connection to GET /api/stream/[sessionId]?product=...&tier1=...

Real-time agent feed renders: Question Agent > Crawler Agent > Document Agent

On 'done' event, researchDoc written to store. User auto-navigated to /research/[sessionId].

/research/[sessionId]

8-section research document with sidebar navigation, citations panel, and download buttons (PDF via @react-pdf/renderer, PPTX via PptxGenJS). Deck config available at /deck-config/[sessionId].

/deck-config/[sessionId]

Configure deck: slide count, tone (investor/casual/technical), focus areas, theme, chart toggle. On generate, POST to /api/deck with researchDoc sections and config. DeckData saved to store, user pushed to /deck/[sessionId].

/deck/[sessionId]

Interactive slide viewer - slide thumbnails sidebar, SlideCanvas renders each DeckSlide type (cover, bullets, features, pricing, gtm, techstack, competitive, stats, funding, sources). PPTX download via /api/pptx.

04

Agent Pipeline & Data Flow

The core pipeline runs server-side as a single long-lived SSE connection. The client opens GET /api/stream/[sessionId] and receives a stream of JSON events. There is no separate orchestrator in Phase 1 - the stream route sequentially invokes each agent using the Vercel AI SDK's generateText function. Each agent step sends progress events before and after execution.

> In DEMO_MODE=true: the stream replays pre-scripted mock events with realistic sleep() delays using getMockResearchDoc(productName). No API calls are made. In live mode, all three agents call the Anthropic API sequentially.

SSE Event Types

Event type	Payload fields	Purpose
agent	agent, status, message	Agent status update (running/done/error)
sources	crawled, total	Progress counter for web crawl
crawl	url, message	Individual URL fetched by Crawler
preview	text	First 2 paragraphs of Executive Summary
done	document: ResearchDoc	Final JSON - triggers navigation to /research
error	message	Pipeline failure, displayed in agent feed

Agent Details

Agent 1 . Question Agent

Model: claude-haiku-4-5-20251001 | Max tokens: 300

Validates that the input is a real software product. Returns 2 sentences: confirmation of existence + category, and core value proposition. Output is used as the first status message in the agent feed.

Agent 2 . Crawler Agent

Model: claude-haiku-4-5-20251001 | Max tokens: 4000 | Tool: webSearch_20250305 (maxUses: 5)

Searches the web using Anthropic's built-in web search tool. Research targets: product overview, pricing, user reviews (G2, Reddit, App Store), funding history, competitors, technical architecture, community metrics. URLs extracted from the response via regex and streamed as 'crawl' events.

Agent 3 . Document Agent

Model: claude-sonnet-4-6 | Max tokens: 8000

Takes the full Crawler research text + Tier 1 user preferences (dimensions, goal, depth) and synthesizes a structured 8-section ResearchDoc JSON. Sections: exec_summary, product_ux, business_model, gtm_growth, tech_arch, market_comp, community, financials. Each section has content (multi-paragraph), keyInsight, and optional stats array.

Agent 4 . Deck Agent (POST /api/deck)

Model: claude-haiku-4-5-20251001 | Max tokens: 4000

Triggered separately from the deck-config page. Receives research section summaries + DeckConfig (slideCount, tone, focus, theme, charts) and generates DeckData JSON - an array of typed slides: cover, bullets, features, pricing, gtm, techstack, competitive, stats, funding, sources.

05

Data Models & Interfaces

All shared types are defined in `src/types/teardown.ts`. The Zustand session store (`src/store/session.ts`) holds the full session state in memory and persists it to `localStorage`.

Core TypeScript Interfaces

ResearchSection

- `id`: string - unique identifier (`exec_summary`, `product_ux`, etc.)
- `title`: string - display heading
- `content`: string - multi-paragraph text, paragraphs separated by `\n\n`
- `keyInsight?`: string - strategic one-liner rendered in a highlighted box
- `stats?`: `Array<{ label, value, sub? }>` - metric cards shown below content

TeardownSource

- `num`: number - citation number [1], [2], ...
- `domain`: string - e.g. `'techcrunch.com'`
- `title`: string - page title
- `url`: string - full URL
- `usedIn`: string - which section referenced this source

ResearchDoc

- `sections`: `ResearchSection[]` - always 8 sections in Phase 1
- `sources`: `TeardownSource[]` - all web sources cited

DeckSlide

- `type`: `'cover' | 'bullets' | 'features' | 'pricing' | 'gtm' | 'techstack' | 'competitive' | 'stats' | 'funding' | 'sources'`
- `title`, `subtitle`, `sectionNum`: string fields
- `bullets?`: `Array<{ text, sub? }>` - for bullet-type slides
- `stats?`: `Array<{ label, value }>` - metric grids
- `items?`: `Array<{ name, desc }>` - for features/techstack
- `tiers?`: `Array<{ name, price, target, highlight? }>` - pricing table
- `competitors?`: `Array<{ name, angle, threat }>` - competitive analysis
- `rounds?`: `Array<{ round, year, amount, lead }>` - funding history

SessionState (Zustand)

- `sessionId`: string - UUID generated on first load
- `productName`: string - entered on landing page
- `tier1Answers`: `{ dimensions[], goal, depth } | null`
- `agentStatuses`: `Record<string, 'idle' | 'running' | 'done' | 'error'>`
- `researchDoc`: `ResearchDoc | null` - populated on SSE 'done' event
- `deckData`: `DeckData | null` - populated after `/api/deck` call

06

API Routes

Method	Route	Description
GET	/api/stream/[sessionId]	Main SSE pipeline. Accepts ?product=&tier1= query params. Runs Question > Crawler > Document agents and streams events.
POST	/api/deck	Deck generation. Body: { productName, sections[], config? }. Returns DeckData JSON.
POST	/api/pptx	PPTX download. Generates .pptx file from DeckData using PptxGenJS.
POST	/api/agents/orchestrator	Placeholder - TODO in Phase 1
POST	/api/agents/question	Placeholder - TODO in Phase 1
POST	/api/agents/crawler	Placeholder - TODO in Phase 1
POST	/api/agents/document	Placeholder - TODO in Phase 1
POST	/api/agents/backend	Placeholder - TODO in Phase 1
POST	/api/agents/pptx	Placeholder - TODO in Phase 1

07

Database Schema (Supabase / PostgreSQL)

Five tables are defined. In Phase 1, Supabase persistence is wired but the primary data flow runs through client-side Zustand state. The database is used for audit logs and future multi-session retrieval.

sessions	session_id UUID PK . product_name TEXT . created_at TIMESTAMPTZ
teardown_research	id UUID PK . session_id FK . sections JSONB . sources JSONB . created_at TIMESTAMPTZ
teardown_qa	id UUID PK . session_id FK . tier1_answers JSONB . tier2_questions JSONB . created_at TIMESTAMPTZ
teardown_pptx	id UUID PK . session_id FK . deck_data JSONB . config JSONB . created_at TIMESTAMPTZ
agent_logs	id UUID PK . session_id FK . agent TEXT . status TEXT . message TEXT . created_at TIMESTAMPTZ

08

Component Architecture

Components follow Next.js App Router conventions: layout.tsx wraps all pages with global fonts and CSS. Route-specific pages are in src/app/. Shared UI is in src/components/ organized by feature.

src/app/

layout.tsx	Root layout - DM Sans + Lora fonts, globals.css
page.tsx	Landing page - Navbar + HeroSection + FooterStrip
intake/page.tsx	Tier 1 Q&A - 3-step wizard, IntakeNav, ProductCard, DimensionGrid, OptionGrid, IntakeFooter
pipeline/[sessionId]/page.tsx	SSE stream page - AgentFeed, PipelineNav, TeardownPreview
research/[sessionId]/page.tsx	Research doc - ResearchNav, SectionSidebar, DocumentBody, CitationsPanel
deck-config/[sessionId]/page.tsx	Deck config form - slide count, tone, focus, theme, chart toggle
deck/[sessionId]/page.tsx	Slide viewer - DeckNav, SlideThumbnails, SlideCanvas

src/components/landing/

Navbar.tsx	Top navigation bar
HeroSection.tsx	Product search hero with animated prompt
FooterStrip.tsx	Bottom footer strip

src/components/intake/

IntakeNav.tsx	Step progress indicator
ProductCard.tsx	Selected product display card
DimensionGrid.tsx	Multi-select grid for research dimensions
OptionGrid.tsx	Single-select grid for goal/depth options
IntakeFooter.tsx	Back/Continue navigation

src/components/pipeline/

AgentFeed.tsx	Real-time SSE event renderer - agent cards with status badges
PipelineNav.tsx	Pipeline page navigation
TeardownPreview.tsx	Live preview of first research paragraphs

src/components/research/

ResearchNav.tsx	Research page navigation + download buttons
SectionSidebar.tsx	Jump-to-section sidebar
DocumentBody.tsx	Renders ResearchSection[] with stats, insights
CitationsPanel.tsx	Sources list with domain links

src/components/deck/

DeckNav.tsx	Deck page navigation + PPTX download
SlideThumbnails.tsx	Left sidebar slide picker
SlideCanvas.tsx	Renders each DeckSlide type as a visual slide

09

Environment Variables & Configuration

Variable	Status	Description
ANTHROPIC_API_KEY	Required	Claude API key for all AI agent calls
NEXT_PUBLIC_SUPABASE_URL	Required	Supabase project URL (browser-safe)
NEXT_PUBLIC_SUPABASE_ANON_KEY	Required	Supabase anon/publishable key (browser-safe)
SUPABASE_SERVICE_ROLE_KEY	Required	Supabase service role key (server-only, never exposed to client)
NEXT_PUBLIC_APP_URL	Optional	App base URL, defaults to http://localhost:3000
NEXT_PUBLIC_DEMO_MODE	Optional	Set to 'true' to bypass all API calls and use mock data

10

Key Libraries & Versions

Next.js 14.2.35	React 18	TypeScript 5	Tailwind CSS 3.4	Zustand 5.0	@ai-sdk/anthropic 3
ai (Vercel) 6	@supabase/supabase-js 2	tRPC 11	@tanstack/react-query 5	@react-pdf/renderer 4	
PptxGenJS 4	Recharts 3	Chart.js 4	Framer Motion 12	Lucide React 1	

11

Deployment & Infrastructure

The application is designed for deployment on Vercel. API routes run as serverless functions. The SSE stream route (/api/stream) uses Node.js runtime (not Edge) to support long-running connections required by the multi-step agent pipeline. The X-Accel-Buffering: no header prevents reverse proxy buffering of the SSE stream.

- Vercel** Serverless functions, automatic scaling, preview deployments per branch
- Node.js 18+** Runtime for all API routes
- Supabase** Managed PostgreSQL - anon key for browser access, service role key for admin writes
- Anthropic API** claude-haiku-4-5 (fast, low-cost) for validation/crawl/deck . claude-sonnet-4-6 (high quality) for document synthesis

Generated from Project_tear codebase . June 2026 . Tear - AI-Powered Product Analysis Platform